

TypeScript Development Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Course Overview

This TypeScript course is designed to take learners from JavaScript fundamentals to advanced, production-ready TypeScript development.

The syllabus covers the TypeScript type system, functions, objects, interfaces, type aliases, classes, generics, utility types, narrowing, modules, configuration, asynchronous programming, API typing, testing, Node.js integration, React integration, code quality, architecture, debugging, and project development.

The course is suitable for students, frontend developers, backend developers, full-stack developers, and working professionals who want to write safer, scalable, and maintainable JavaScript applications using TypeScript.

1. Prerequisites

Before starting TypeScript, learners should understand:

JavaScript Fundamentals

- Variables
- Data types
- Operators
- Conditional statements
- Loops
- Functions
- Arrays
- Objects
- Classes
- Scope
- Closures
- DOM basics
- Events
- Error handling

Modern JavaScript

- let
- const
- Arrow functions
- Template literals
- Destructuring
- Spread operator
- Rest parameters
- Modules
- Promises
- Async/await

2. Introduction to TypeScript

What is TypeScript?

- Definition
- History
- Relationship with JavaScript
- TypeScript as a superset of JavaScript

Why TypeScript?

- Static type checking
- Better IDE support
- Refactoring safety
- Maintainability
- Large application development

TypeScript vs JavaScript

- Compilation
- Type checking
- Developer experience
- Runtime behavior

Common Use Cases

- Frontend applications
- Backend applications
- Full-stack development
- Libraries
- Enterprise software

3. TypeScript Development Environment

Required Tools

- Node.js
- npm
- TypeScript compiler
- VS Code
- Git
- GitHub

Installing TypeScript

- Global installation
- Local project installation

TypeScript Compiler

- `tsc`
- Checking compiler version
- Compiling TypeScript

Running TypeScript

- Compile then execute
- Development runners
- `tsx`
- `ts-node` concepts

4. Creating a TypeScript Project

Project Initialization

- `npm init`
- Installing TypeScript
- Creating source directories

Creating `tsconfig.json`

- `tsc --init`

Suggested Project Structure

- src
- dist
- tests
- types
- config

Build Scripts

- Development
- Type checking
- Production build

5. Basic Types

Primitive Types

- string
- number
- boolean
- bigint
- symbol

Special Types

- null
- undefined

Type Annotations

- Variable annotations
- Explicit typing

Type Inference

- Automatic type detection
- When annotation is unnecessary

6. any, unknown, never, and void

any

- Disabling type checking
- Risks of excessive any

unknown

- Safer alternative to any
- Narrowing before use

void

- Functions without meaningful return values

never

- Functions that never return
- Exhaustive type checking

Choosing the Correct Type

7. Arrays

Typed Arrays

- `number[]`
- `string[]`

Generic Array Syntax

- `Array<number>`

Mixed Arrays

- Union arrays

Readonly Arrays

- `readonly`
- `ReadonlyArray`

Array Method Typing

- `map`
- `filter`
- `reduce`
- `find`

8. Tuples

Tuple Fundamentals

- Fixed-length arrays
- Typed positions

Optional Tuple Elements

Rest Elements in Tuples

Readonly Tuples

Named Tuples

Tuple Use Cases

9. Object Types

Defining Object Shapes

- Inline object types

Required Properties

Optional Properties

Readonly Properties

Nested Objects

Object Type Inference

10. Functions

Function Parameter Types

Return Types

Optional Parameters

Default Parameters

Rest Parameters

Function Expressions

Arrow Functions

Callback Functions

11. Function Types

Defining Function Signatures

Type Aliases for Functions

Interfaces for Callable Types

Callback Type Safety

Higher-Order Functions

Function Overloads

12. Union Types

Creating Union Types

Primitive Unions

Object Unions

Function Parameters with Unions

Narrowing Union Types

Practical Use Cases

13. Literal Types

String Literal Types

Number Literal Types

Boolean Literal Types

Literal Unions

Status Types

- "pending"
- "success"
- "failed"

Safer Constants

14. Type Aliases

Creating Type Aliases

Primitive Aliases

Object Aliases

Union Aliases

Function Aliases

Generic Type Aliases

Reusing Types

15. Interfaces

Interface Fundamentals

Defining Object Shapes

Optional Properties

Readonly Properties

Methods

Extending Interfaces

Interface Merging

16. Interface vs Type Alias

Similarities

Differences

Declaration Merging

Union Support

Intersection Support

When to Use Interfaces

When to Use Type Aliases

17. Intersection Types

Combining Types

Multiple Interfaces

Intersection with Type Aliases

Practical Use Cases

Conflicting Properties

18. Enums

Numeric Enums

String Enums

Computed Members

Enum Use Cases

`const enum` Concepts

Alternatives to Enums

- Literal unions
- `as const`

19. Type Assertions

Type Assertion Syntax

as Syntax

Angle Bracket Syntax

Double Assertions

Risks of Incorrect Assertions

Safer Alternatives

20. `as const`

Literal Preservation

Readonly Object Properties

Readonly Arrays

Constant Configuration Objects

Deriving Union Types from Constants

21. Type Narrowing

What is Narrowing?

`typeof`

Equality Checks

Truthiness Checks

`in` Operator

`instanceof`

Control Flow Analysis

22. Type Guards

Built-In Type Guards

User-Defined Type Guards

Type Predicate Syntax

Guard Functions

Reusable Validation Helpers

23. Discriminated Unions

Discriminator Properties

Safe Branching

Switch Statements

State Modeling

API Response Modeling

UI State Modeling

24. Exhaustive Checking

never

Exhaustive switch

Detecting Unhandled Cases

Benefits in Large Applications

25. Classes

Class Fundamentals

Properties

Constructors

Methods

Creating Objects

Class Type Inference

26. Access Modifiers

public

private

protected

ECMAScript Private Fields

Readonly Properties

Encapsulation

27. Inheritance

extends

Base Classes

Derived Classes

super

Method Overriding

Polymorphism

28. Abstract Classes

abstract **Classes**

Abstract Methods

Concrete Methods

Shared Base Logic

Use Cases

29. Interfaces with Classes

Implementing Interfaces

Multiple Interfaces

Contract-Based Design

Dependency Abstraction

30. Static Members

Static Properties

Static Methods

Utility Classes

Class-Level State

31. Getters and Setters

Getter Methods

Setter Methods

Validation

Encapsulation

Computed Properties

32. Generics

Why Generics?

Generic Functions

Generic Types

Generic Interfaces

Generic Classes

Generic Type Aliases

33. Generic Constraints

extends Constraints

Constraining Object Shapes

keyof Constraints

Multiple Generic Parameters

Default Generic Types

34. keyof

Key Extraction

Object Property Unions

Generic Property Access

Type-Safe Utility Functions

35. typeof in Type Positions

Deriving Types from Variables

Deriving Types from Objects

Deriving Types from Functions

Combining typeof with keyof

36. Indexed Access Types

Property Type Lookup

Nested Type Access

Array Element Types

Reusing Existing Types

37. Conditional Types

Basic Conditional Types

Generic Conditional Logic

Distribution over Unions

Practical Use Cases

Reusable Type Utilities

38. infer

Type Inference Inside Conditional Types

Extracting Return Types

Extracting Promise Values

Extracting Array Element Types

Advanced Generic Utilities

39. Mapped Types

Mapping Over Keys

Making Properties Optional

Making Properties Readonly

Removing Modifiers

Custom Utility Types

40. Template Literal Types

String-Based Type Construction

Combining Literal Types

Event Name Types

Route Types

Dynamic Property Names

41. Utility Types

Partial<T>

Required<T>

Readonly<T>

Pick<T, K>

Omit<T, K>

Record<K, T>

Exclude<T, U>

Extract<T, U>

NonNullable<T>

42. Function Utility Types

Parameters<T>

ReturnType<T>

ConstructorParameters<T>

InstanceType<T>

ThisParameterType<T>

OmitThisParameter<T>

43. Nullable Types

null

undefined

Optional Properties

Optional Parameters

Strict Null Checking

Nullish Coalescing

Optional Chaining

44. Index Signatures

Dynamic Object Keys

String Index Signatures

Number Index Signatures

Readonly Index Signatures

Limitations and Risks

45. Record and Dictionary Types

Dictionary Objects

Key-Value Maps

Typed Lookup Tables

Configuration Maps

Status Mappings

46. Modules

ES Modules

Named Exports

Default Exports

Imports

Re-Exports

Barrel Files

47. Module Resolution

Relative Imports

Absolute Imports

Path Aliases

Node Module Resolution Concepts

ESM vs CommonJS Concepts

Package Exports

48. Namespaces

Namespace Fundamentals**Internal Modules****Legacy Use Cases****Namespaces vs ES Modules****When to Avoid Namespaces****49. Declaration Files****.d.ts Files****Purpose of Declaration Files****Declaring Modules****Global Declarations****Library Type Definitions****DefinitelyTyped****50. Third-Party Library Typings****Installing @types/*****Libraries with Built-In Types****Missing Type Definitions****Declaring Untyped Packages****Extending Existing Type Definitions****51. Declaration Merging****Interface Merging****Namespace Merging****Module Augmentation****Extending Library Types****Risks and Best Practices****52. tsconfig.json****Compiler Configuration****Common Options**

- target
- module
- moduleResolution
- strict
- rootDir
- outDir

Source Maps

Declaration Output**Include and Exclude****53. Strict Mode**`strict``strictNullChecks``noImplicitAny``strictFunctionTypes``strictPropertyInitialization`**Why Strict Mode Matters****54. Additional Compiler Safety Options**`noUncheckedIndexedAccess``exactOptionalPropertyTypes``noImplicitOverride``noFallthroughCasesInSwitch``noUnusedLocals``noUnusedParameters`**Choosing Team Standards****55. Async Programming****Promises**`Promise<T>`**Async Functions****Await****Typed Promise Results****Promise Rejection Handling****56. Concurrent Async Operations**`Promise.all``Promise.allSettled``Promise.race``Promise.any`**Typing Concurrent Results****57. Error Handling**

try

catch

finally

Unknown Errors

Narrowing Error Objects

Custom Error Classes

58. Custom Error Types

Extending Error

Domain Errors

Validation Errors

HTTP Errors

Error Codes

Error Handling Strategies

59. DOM with TypeScript

DOM Element Types

querySelector

getElementById

HTML Element Types

Event Types

Null Safety

60. Event Typing

Mouse Events

Keyboard Events

Input Events

Form Events

Custom Events

Event Listener Typing

61. JSON and External Data

JSON Parsing

Type Assertions and Risks

Runtime Validation

API Data Safety

Schema Validation Concepts

62. Runtime Validation

TypeScript types do not validate runtime data.

Why Runtime Validation Matters

Validation Libraries

- Zod
- Valibot
- Yup concepts

Validating API Responses

Validating Form Data

Inferring TypeScript Types from Schemas

63. REST API Typing

Request Types

Response Types

DTOs

Error Response Types

Pagination Types

Query Parameter Types

Reusable API Contracts

64. Fetch API with TypeScript

Typed Fetch Functions

Generic API Clients

Request Configuration

JSON Responses

Error Handling

AbortController

65. Axios with TypeScript

Axios Setup

Typed Responses

Request Types

Axios Instances

Interceptors

Typed Error Handling

66. Backend Development with TypeScript

Node.js with TypeScript

Project Setup

Environment Variables

File System APIs

HTTP Servers

Async Code

67. Express with TypeScript

Express Setup

Request Types

Response Types

Route Parameters

Query Parameters

Middleware Typing

Error Middleware

68. Fastify / NestJS Concepts

Framework Overview

TypeScript-First Backend Development

Controllers

Services

Dependency Injection

DTOs

Validation

69. React with TypeScript

Typing Components

Props

Optional Props

Children

Event Handlers

State

Refs

Context

70. React Hooks with TypeScript

useState

useRef

useReducer

useContext

useCallback

useMemo

Custom Hooks

71. Forms with TypeScript

Controlled Inputs

Form Event Types

Form Data Models

Validation Models

React Hook Form Concepts

Schema-Based Validation

72. Next.js with TypeScript

Typed Pages and Layouts

Route Parameters

Search Parameters

Server Components

Route Handlers

Server Actions

API Models

73. Angular and TypeScript

Why Angular Uses TypeScript

Components

Services

Dependency Injection

Interfaces

Generics

RxJS Typing

74. TypeScript with Databases

Database Models

ORM-Generated Types

Prisma Concepts

Drizzle Concepts

Typed Queries

Nullable Database Fields

75. Type-Safe Environment Variables

`process.env`

Problems with Untyped Configuration

Runtime Validation

Environment Schemas

Development vs Production Variables

76. Type-Safe Configuration

Configuration Interfaces

Readonly Configuration

Literal Configuration Values

Validation

Centralized Config Modules

77. Functional Programming with TypeScript

Pure Functions

Immutability

Higher-Order Functions

Function Composition

Readonly Types

Result Types

78. Result and Error Types

Success/Failure Modeling

Discriminated Unions

Result Pattern

Avoiding Exceptions for Expected Errors

API Result Types

79. Domain Modeling

Modeling Business Rules with Types

Entity Types

Value Objects

State Machines

Discriminated Unions

Making Invalid States Harder to Represent

80. Advanced Domain Types

Branded Types

Nominal-Typing Patterns

ID Types

Validated String Types

Currency Types

Date Types

81. Immutability

readonly

Readonly Arrays

Readonly Tuples

Readonly<T>

Immutable Update Patterns

Benefits in State Management

82. Design Patterns with TypeScript

Factory Pattern

Strategy Pattern

Observer Pattern

Repository Pattern

Adapter Pattern

Dependency Injection

Builder Pattern

83. SOLID Principles

Single Responsibility Principle

Open/Closed Principle

Liskov Substitution Principle

Interface Segregation Principle

Dependency Inversion Principle

Applying SOLID with TypeScript

84. Clean Architecture Concepts

Domain Layer

Application Layer

Infrastructure Layer

Presentation Layer

Dependency Direction

Type Contracts Between Layers

85. Code Organization

Feature-Based Structure

Layer-Based Structure

Shared Types

Domain Types

API Types

Avoiding Large Global Type Files

86. Type Design Best Practices

Prefer Inference When Clear

Avoid any

Prefer unknown for External Data

Use Discriminated Unions

Keep Types Close to Domain Logic

Avoid Overly Complex Generic Types

87. Generic Design Best Practices

Keep Generic APIs Understandable

Use Constraints

Use Meaningful Generic Names

Avoid Unnecessary Generics

Prefer Concrete Types When Appropriate

88. Code Quality Tools

ESLint

TypeScript ESLint

Prettier

Editor Integration

Automated Formatting

Pre-Commit Hooks Concepts

89. Linting TypeScript

Type-Aware Linting

Unsafe any

Floating Promises

Unused Variables

Explicit Return Types

Consistent Type Imports

90. Testing TypeScript Applications

Testing Fundamentals

Unit Testing

Integration Testing

End-to-End Testing

Test Types

Test Coverage

91. Vitest and Jest

Test Suites

Test Cases

Assertions

Mock Functions

Spies

Async Tests

Type-Safe Tests

92. Testing Typed APIs

Mock API Responses

Request Validation Tests

Error Responses

Contract Tests Concepts

Schema Validation Tests

93. Testing React TypeScript Applications

React Testing Library

Component Props

Events

Forms

Async UI

Typed Test Utilities

94. Debugging

Compiler Errors

Runtime Errors

Source Maps

VS Code Debugging

Browser Debugging

Node.js Debugging

95. Understanding Compiler Errors

Reading Error Messages

Type Mismatch Errors

Missing Properties

Generic Constraint Errors

Union Narrowing Errors

Debugging Complex Types

96. Performance Considerations

Compile-Time Types vs Runtime

Type Erasure

Build Performance

Incremental Compilation

Project References Concepts

Avoiding Type-Level Overengineering

97. Large TypeScript Projects

Project Structure

Shared Packages

Monorepos

Path Aliases

Project References

Build Pipelines

Dependency Boundaries

98. Monorepos

Monorepo Concepts

npm Workspaces

pnpm Workspaces

Turborepo Concepts

Nx Concepts

Shared Type Packages

99. Library Development

Creating TypeScript Libraries

Public APIs

Declaration Files

Build Output

ESM and CommonJS

Semantic Versioning

Publishing Concepts

100. Package Typing

package.json Types

types

exports

Conditional Exports

Declaration Maps

Consumer Compatibility

101. API Contract Sharing

Shared Request/Response Types

Frontend/Backend Contracts

Risks of Shared Domain Models

Schema-First Contracts

OpenAPI-Generated Types Concepts

GraphQL-Generated Types Concepts

102. OpenAPI with TypeScript

OpenAPI Fundamentals

Generating Types

API Client Generation

Contract Synchronization

Reducing Frontend/Backend Drift

103. GraphQL with TypeScript

GraphQL Types

Queries

Mutations

Schema Types

Generated Client Types

Type-Safe GraphQL Development

104. WebSockets with TypeScript

WebSocket Events

Typed Messages

Client/Server Contracts

Discriminated Message Types

Real-Time Application Patterns

105. Security Considerations

Type Safety Is Not Security

Input Validation

Authorization Types

Avoiding Sensitive Data Exposure

Runtime Validation

Dependency Security

106. AI-Assisted TypeScript Development

Using AI Coding Assistants

- Code generation
- Refactoring
- Type generation
- Explaining compiler errors
- Writing tests

AI for Type Design

- Converting JSON into interfaces
- Creating schemas
- Generating discriminated unions

AI Code Review

- Detecting any
- Checking unsafe assertions
- Reviewing generics
- Reviewing runtime validation

Responsible Usage

- Verify generated code
- Avoid outdated APIs
- Protect confidential code

107. Mini Project 1 – TypeScript Utility Library

Features

- String utilities
- Array utilities
- Object utilities
- Generic functions
- Unit tests

Concepts

- Functions
- Generics
- Utility types
- Modules

108. Mini Project 2 – Task Manager

Features

- Create tasks
- Update tasks
- Delete tasks
- Filter by status

Concepts

- Interfaces
- Literal types
- Discriminated unions
- Arrays
- Functions

109. Mini Project 3 – Expense Tracker

Features

- Add expenses
- Categories
- Totals
- Reports

Concepts

- Classes
- Interfaces
- Generics
- Date types
- Utility types

110. Mini Project 4 – REST API Client

Features

- Generic HTTP client
- GET
- POST
- PUT
- DELETE
- Error handling

Concepts

- Generics
- Async/await
- API types
- Error models

111. Mini Project 5 – Node.js REST API

Features

- CRUD endpoints
- Validation
- Error handling
- Typed requests and responses

Concepts

- Node.js
- Express/Fastify
- DTOs
- Middleware
- Runtime validation

112. Mini Project 6 – React TypeScript Application

Features

- Components
- Forms
- API integration
- State
- Routing

Concepts

- Props
- Hooks
- Events
- API models
- Generics

113. Mini Project 7 – Type-Safe Dashboard**Features**

- Tables
- Filters
- Sorting
- Pagination
- API calls

Concepts

- Generics
- URL types
- API models
- Utility types

114. Major Capstone Project – Full-Stack TypeScript Application**Suggested Application**

Build a complete business application using TypeScript on both frontend and backend.

Example Features

- Authentication
- User management
- CRUD operations
- Search
- Filters
- Pagination
- Role-based access
- API integration
- Database integration
- Validation
- Testing

Suggested Stack

- TypeScript
- React or Next.js
- Node.js
- Express/Fastify/NestJS
- PostgreSQL or MongoDB
- Zod
- Vitest/Jest
- Playwright

115. Capstone Project Requirements

The final project should demonstrate:

- Strict TypeScript configuration
- Minimal or zero unnecessary any
- Interfaces and type aliases
- Generics
- Utility types
- Discriminated unions
- Runtime validation
- Typed API contracts
- Error handling
- Asynchronous programming
- Modular architecture
- Testing
- ESLint
- Prettier
- Git/GitHub
- Documentation
- Deployment

116. Interview Preparation – Core TypeScript

Important Topics

- TypeScript vs JavaScript
- Type inference
- any vs unknown
- never
- void
- Interfaces vs type aliases
- Union types
- Intersection types
- Literal types
- Generics
- keyof
- typeof
- Utility types
- Type narrowing
- Type guards

117. Interview Preparation – Advanced TypeScript

Advanced Topics

- Conditional types
- infer
- Mapped types
- Indexed access types
- Template literal types
- Declaration files
- Module augmentation
- Generic constraints
- Exhaustive checking
- Branded types

118. Practical Interview Exercises

Candidates should be able to:

- Convert JavaScript to TypeScript
- Create interfaces from JSON
- Type a generic function
- Implement a type guard
- Model API responses
- Build a discriminated union
- Remove any from existing code
- Write a generic API client
- Type a React component
- Debug a compiler error

119. Coding Exercises

Students should practice:

- Typed calculator
- Generic array utilities
- Object mapper
- Safe property accessor
- Event emitter
- Result type
- Typed API client
- Pagination model
- Search filter types
- Form model
- Validation schema
- Generic repository
- State reducer
- Branded ID type
- Exhaustive switch

120. Recommended Modern TypeScript Stack

A modern TypeScript development stack may include:

- TypeScript
- Node.js
- npm / pnpm
- VS Code
- ESLint
- TypeScript ESLint
- Prettier
- Vitest or Jest
- Zod or another schema validation library
- React / Next.js for frontend development
- Node.js / Express / Fastify / NestJS for backend development
- Git
- GitHub

121. Suggested Course Sequence

Phase 1 – JavaScript Foundation

- JavaScript refresher

- ES6+ features
- Modules
- Async programming

Phase 2 – TypeScript Fundamentals

- TypeScript setup
- Basic types
- Arrays and tuples
- Objects
- Functions
- Interfaces
- Type aliases
- Union and literal types

Phase 3 – Intermediate TypeScript

- Classes
- Generics
- keyof
- Type guards
- Narrowing
- Utility types
- Modules
- Compiler configuration

Phase 4 – Advanced Type System

- Conditional types
- infer
- Mapped types
- Indexed access
- Template literal types
- Declaration files
- Module augmentation

Phase 5 – Application Development

- Async TypeScript
- REST API typing
- Runtime validation
- Node.js
- Express/Fastify
- React
- Next.js

Phase 6 – Professional TypeScript

- Architecture
- SOLID
- Design patterns
- Testing
- Linting
- Debugging
- Monorepos

Phase 7 – Career Preparation

- Mini projects
- Capstone project

- GitHub portfolio
- Interview preparation
- Mock interviews

122. Learning Outcomes

After completing this TypeScript course, learners should be able to:

- Understand the TypeScript type system
- Add TypeScript to JavaScript projects
- Use type inference and annotations effectively
- Work with interfaces and type aliases
- Model data using unions and discriminated unions
- Build type-safe functions
- Use classes and object-oriented concepts
- Build generic reusable utilities
- Use TypeScript utility types
- Create conditional and mapped types
- Perform safe type narrowing
- Work with declaration files
- Configure TypeScript projects
- Build typed asynchronous applications
- Create type-safe REST APIs
- Validate runtime data
- Build frontend applications using TypeScript
- Build backend applications using TypeScript
- Write unit and integration tests
- Apply clean architecture and SOLID concepts
- Build portfolio-ready projects
- Prepare for TypeScript interviews

123. Suggested Course Deliverables

By the end of the course, each learner should ideally have:

- TypeScript coding exercises
- Type-system assignments
- Generics assignments
- API typing exercises
- Runtime validation exercises
- 5–7 mini projects
- One full-stack capstone project
- GitHub repositories
- Professional README files
- Automated tests
- ESLint/Prettier configuration
- Interview question preparation
- Mock interview experience
- AI-assisted coding practice
- A TypeScript development portfolio

124. Optional Advanced Modules

Type-Level Programming

- Recursive conditional types

- Recursive mapped types
- Complex inference patterns

Compiler API Concepts

- AST
- Programmatic type analysis
- Code transformation

TypeScript Transformers

- Compiler extension concepts

Code Generation

- Schema-to-TypeScript
- OpenAPI-to-TypeScript
- GraphQL code generation

Advanced Library Development

- Declaration publishing
- Multi-runtime packages
- ESM/CJS interoperability

125. Career Roles After TypeScript Training

This course can help learners prepare for roles such as:

- TypeScript Developer
- JavaScript Developer
- Frontend Developer
- React Developer
- Next.js Developer
- Angular Developer
- Node.js Developer
- Full-Stack Developer
- Web Application Developer
- Software Engineer

Conclusion

TypeScript has become a core technology for building modern JavaScript applications at scale. A strong TypeScript developer should understand much more than basic type annotations—they should know type inference, interfaces, unions, generics, utility types, type narrowing, advanced type operators, runtime validation, modules, configuration, testing, architecture, and framework integration.

This syllabus provides a structured path from JavaScript fundamentals to advanced TypeScript development and prepares learners for frontend, backend, and full-stack TypeScript roles.

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.